

STOCKFLOW

Documentacao do prototipo - Sistema de Controle de Estoque

Python + Tkinter + MySQL + Matplotlib

Objetivo

O StockFlow e um prototipo desktop criado para demonstrar um sistema simples de controle de estoque. O projeto concentra cadastro de produtos, movimentacoes, indicadores e grafico em uma aplicacao pequena e facil de explicar.

1. Introducao

A aplicacao possui uma janela de login e uma janela principal. Depois da autenticao, o usuario pode consultar o dashboard e realizar o CRUD de produtos. O historico e automatico: o cadastro registra a quantidade inicial como entrada, a edicao registra apenas a diferenca como entrada ou saida, e a exclusao registra a quantidade restante como saida. O historico continua visivel mesmo depois da exclusao do produto.

2. Atendimento aos criterios do trabalho

Criterio	Como foi atendido
Multiplas janelas	Login, janela principal e formularios secundarios para cadastro/edicao.
MySQL	DatabaseManager cria/acessa o banco estoque_db e trabalha com usuarios, produtos e movimentacoes.
CRUD completo	Create, Read, Update e Delete implementados para produtos.
Grafico	Matplotlib gera um grafico de barras com a quantidade atual de cada produto.
Log de erros	Logger registra execucoes em error.log com data, hora e detalhes.
POO	Classes DatabaseManager, LoginWindow, MainWindow e Logger organizam as responsabilidades.
Biblioteca extra	Matplotlib e mysql-connector-python.
Documentacao	Este PDF explica objetivo, arquitetura, banco, execucao e demonstracao.

3. Arquitetura do projeto

A aplicacao separa responsabilidades em modulos. main.py inicia o sistema. LoginWindow cuida da autenticao. MainWindow concentra a interface e as funcionalidades. DatabaseManager concentra as operacoes no MySQL. Logger centraliza o registro de erros.

Arquivo / classe	Responsabilidade
main.py	Inicializar Logger, banco e janela de login.
database.py / DatabaseManager	Conexao, criacao das tabelas, CRUD, movimentacoes e consultas do dashboard.
login.py / LoginWindow	Formulario de login e validacao de usuario e senha.
gui.py / MainWindow	Dashboard, produtos, formularios e historico de movimentacoes.
logger.py / Logger	Registro das excecoes no arquivo error.log.

4. Banco de dados

Tabelas utilizadas:

usuarios - acesso ao sistema.

produtos - cadastro e saldo atual.

movimentacoes - registro das entradas e saidas.

Relacionamento: movimentacoes.produto_id referencia produtos.id com ON DELETE SET NULL, e produto_nome guarda o nome no momento da movimentacao. Assim, o historico sobrevive a exclusoes. O CRUD registra as mudancas de quantidade dentro da mesma transacao que altera o saldo.

5. CRUD de produtos

Create: cadastra um produto.

Read: lista e consulta produtos.

Update: altera os dados de um produto.

Delete: exclui o produto selecionado.

6. Movimentacoes

Ao cadastrar um produto com quantidade inicial maior que zero, o sistema registra uma ENTRADA. Ao editar, somente a diferenca de quantidade vira ENTRADA ou SAIDA. Ao excluir, a quantidade restante e registrada como SAIDA antes da exclusao. O sistema impede quantidade negativa e o historico mostra as 50 movimentacoes mais recentes.

7. Execucao

1. Ligar o MySQL pelo XAMPP ou servidor local.
2. Criar e ativar a venv.
3. Executar pip install -r requirements.txt.
4. Executar python main.py.

Login inicial: admin / admin123

8. Roteiro de demonstracao

Login -> Dashboard -> Produtos -> cadastrar produto e quantidade -> historico mostra ENTRADA -> editar quantidade para cima/baixo -> historico mostra a diferenca -> excluir produto de teste -> historico mantem a

SAIDA da quantidade restante -> dashboard/grafico -> error.log.

9. Personalizacao visual

As cores principais estao concentradas em THEME, em gui.py. O login possui LOGIN_THEME em login.py. Tambem podem ser alterados nome do sistema, fontes, espacamentos, botoes, tamanho das janelas e logo, mantendo a logica do banco separada.

10. UML simplificado

DatabaseManager fornece os dados para LoginWindow e MainWindow. Logger e utilizado para registrar excecoes. DatabaseManager se comunica com o MySQL. O relacionamento de banco principal e movimentacoes.produto_id -> produtos.id, com nome armazenado no historico para preservar registros de produtos excluidos.